# PLUCK

Floating Clipboard for Windows

**Product Specification Document**

Version 1.0 | May 2025

| | |
|---|---|
| **Project Name** | Pluck |
| **Version** | 1.0 — Initial Release |
| **Platform** | Windows 10 / 11 |
| **Tech Stack** | C# WPF + Win32 P/Invoke (C++/CLI) |
| **License** | MIT Open Source |
| **Author** | [Your Name] |
| **Status** | In Planning |

## 1. Executive Summary

Pluck is a next-generation clipboard manager for Windows that reimagines how users interact with copied content. Instead of hiding clipboard history in a static list, Pluck surfaces each copied item as a floating bubble directly on the screen — always visible, always accessible, and designed to be interacted with through natural drag-and-drop gestures.

Modern power users — developers, designers, content creators — work across dozens of applications and browser tabs simultaneously. The native Windows clipboard handles only one item at a time, forcing users into a repetitive copy-switch-paste loop that breaks focus and wastes time. Pluck solves this by making the clipboard a first-class, always-visible workspace element.

Pluck is built for portfolio showcase and is distributed as a free, open-source project on GitHub under the MIT license. Monetization avenues exist through a freemium model (premium features, cloud sync) and GitHub Sponsors, but user adoption and developer credibility are the primary success metrics for v1.0.

# 2. Problem Statement

## 2.1 Pain Points

- The Windows clipboard stores only one item at a time — any new copy immediately overwrites the previous content.
- Accessing clipboard history (Win+V) opens a static panel that users must navigate separately, interrupting their current focus context.
- There is no visual feedback when a copy action is performed — users have no confirmation of what was captured or from which application.
- Paste operations require the user to locate the target field, switch to the clipboard panel, find the correct item, and then return — a minimum of four steps per paste.
- Screenshots and image content are particularly difficult to manage, as previewing them requires opening a dedicated image viewer.

## 2.2 Target Users

Pluck is designed for Windows power users who regularly work with multiple applications simultaneously. The primary personas are:

- Software Developers — copying code snippets, error messages, API keys, and file paths across IDEs, terminals, and browsers.
- Designers & Content Creators — working with image assets, colour codes, and text across Figma, Photoshop, and Office tools.
- Remote Workers — managing content across video-conferencing tools, project management boards, documentation, and communication platforms.

# 3. Product Overview

## 3.1 Vision

***“Your clipboard, alive on screen.”***

 

Pluck makes copied content a tangible, spatial object — one you can see, move, pin, and act upon without leaving your current task context. It is the clipboard reimagined as a workspace tool rather than a hidden utility.

### 3.2 Core Architecture

Pluck consists of two distinct subsystems that work together:

- Main Dialog — a traditional clipboard history panel with a rich settings interface, accessible from the system tray.
- Bubble Overlay System — a full-screen transparent overlay window that renders all active bubbles and handles user interaction with them.

### 3.3 Key Differentiators

| Feature | Windows Clipboard (Win+V) | Existing Managers (Ditto, CopyQ) | Pluck |
|---|---|---|---|
| Multi-item history | Yes (limited) | Yes | Yes |
| Visual floating UI | No | No | Yes — core feature |
| Source app detection | No | Partial | Yes (icon + name) |
| Drag-to-paste | No | No | Yes |
| Smart format paste | No | No | Yes |
| Pop effect on paste | No | No | Yes |
| Open source (MIT) | N/A | Varies | Yes |

## 4. Functional Requirements

### 4.1 System Tray

- Pluck runs silently in the background and is represented by an icon in the Windows system tray.
- Left-click on the tray icon opens or closes the Main Dialog.
- Right-click on the tray icon opens a context menu with: Open, Settings, Clear All, and Exit.
- A global hotkey (e.g. Ctrl+Shift+V) can open the Main Dialog from any application, configurable in Settings.

## 4.2 Main Dialog

### 4.2.1 Clipboard History Panel

- Displays a scrollable list of all clipboard items captured during the current session.
- Each item in the list shows: a type icon (text, image, file), a content preview, the source application name and icon, and the timestamp of the copy event.
- Items support the following actions via inline buttons or right-click context menu:
  - Paste — pastes the item directly into the currently focused application.
  - Pin — marks the item as protected from automatic deletion when the limit is reached.
  - Delete — removes the item from history immediately.
  - Copy again — re-copies the item to the active clipboard slot.
- The panel supports keyboard navigation (arrow keys to select, Enter to paste, Delete to remove).

### 4.2.2 Settings Panel

The Settings panel is divided into two sections: Bubble Settings and General Settings.

**Bubble Settings:**

- Opacity — slider from 10% to 90%. Values of 0% and 100% are explicitly excluded to ensure bubbles remain usable without fully obscuring the screen.
- Maximum Bubble Count — integer from 10 to 50. When the limit is reached, the oldest unpinned bubble is automatically removed.
- Display Duration — toggle between Off (bubble persists until manually dismissed) and On with a configurable time value in seconds (bubble auto-dismisses after the set duration unless pinned).
- Floating Animation — On/Off toggle. When enabled, bubbles apply a gentle vertical oscillation using a shared sinusoidal TranslateTransform. When disabled, bubbles remain stationary.
- Bubble Content Display — three modes: Best Content (auto-selects the richest preview format), Known Content Only (shows text/image only, skips binary/unknown types), Disabled (shows type icon only).
- Show Source App Icon — On/Off toggle.
- Show Source App Name — On/Off toggle.
- Show Copy Timestamp — On/Off toggle.
- Pop Effect on Paste — On/Off toggle. When enabled, a particle burst animation plays as the bubble is dismissed after a successful paste.

**General Settings:**

- Global Hotkey — configurable key combination to open the Main Dialog.
- Launch at Windows Startup — On/Off toggle.
- History Limit — maximum number of items to retain in the SQLite history database.
- Clear History on Exit — On/Off toggle.

## 4.3 Bubble Overlay System

### 4.3.1 Bubble Creation

- Every time a copy event is detected (via AddClipboardFormatListener), a new bubble is created.
- Bubbles snap to the right edge of the screen by default and are auto-arranged in a vertical stack.
- When the bubble count exceeds the configured maximum, the oldest unpinned bubble is removed before the new one appears.
- When five or more bubbles are stacked, they collapse into a grouped stack indicator showing a count badge. Clicking the stack expands all bubbles.

### 4.3.2 Bubble Appearance

- Each bubble is a rounded card rendered inside a single full-screen transparent overlay window (one HWND for all bubbles — see Technical Architecture).
- Bubble contents (conditionally shown per settings):
  - Source application icon (16x16 or 32x32, extracted from the process executable).
  - Source application name (process display name).
  - Copy timestamp (formatted as HH:mm or MM/DD HH:mm for older items).
  - Content preview — text excerpt (first 60 characters), image thumbnail, or file path.
  - Pin indicator icon (visible when item is pinned).
- Opacity is applied uniformly as configured in Settings.

### 4.3.3 Bubble Interactions

The following interactions are supported on each bubble:

| Interaction | Action | Notes |
|---|---|---|
| Hover | Bubble scales up slightly (1.05x) with a smooth ease-out animation | Duration: ~150ms |
| Left Click | Pastes item into the last focused window | Focus-aware format selection |
| Drag & Drop | Drag bubble onto a target window to paste | Highlights target window on hover |
| Right Click | Opens context menu: Paste, Pin, Copy Again, Delete | — |
| Ctrl + Drag | Repositions bubble on screen | Custom modifier key, configurable |
| Middle Click | Deletes bubble immediately | Shortcut for power users |

#### 4.3.4 Focus-Aware Paste

- When the user drags a bubble towards a target application window, that window is highlighted with a subtle blue border overlay.
- Upon drop, Pluck uses the target window's registered IDropTarget interface to paste in the most appropriate format:
  - Text fields — CF_UNICODETEXT
  - Image-accepting targets (image editors, Slack, Teams) — CF_BITMAP or PNG format
  - File explorer and file-drop targets — CF_HDROP
- If no native format match exists, Pluck falls back to CF_UNICODETEXT (stringified value).

# 5. Technical Architecture

## 5.1 Technology Stack

| Layer | Technology | Rationale |
|---|---|---|
| UI Framework | C# WPF (.NET 8) | Native Windows animations, transparency, XAML-based bubble rendering |
| Performance-Critical Layer | C++/CLI + P/Invoke | Clipboard hook, drag-drop COM interfaces, image memory pool |
| Data Storage | SQLite via Microsoft.Data.Sqlite | Lightweight embedded database, no external dependencies |
| Image Handling | WPF BitmapSource + custom memory pool | Avoids GC pressure from large image allocations |
| Packaging | MSIX / single-file .exe | Clean install/uninstall, no registry pollution |

## 5.2 Project Structure

The solution is organized into three projects:

- Pluck.Core — C++/CLI class library responsible for all Win32 interactions: clipboard hook (AddClipboardFormatListener), source process detection (GetForegroundWindow + QueryFullProcessImageName), drag-and-drop engine (IDropSource / IDropTarget COM implementation), and the image memory pool.
- Pluck.UI — C# WPF application project. Contains the BubbleOverlayWindow (single transparent HWND rendering all bubbles on a WPF Canvas), the MainDialog (history + settings), and the TrayIcon controller.

- Pluck.Data — C# class library containing the ClipboardItem model, the SQLite repository, and history management logic (max item enforcement, pin protection).

## 5.3 Key Technical Decisions

### Single Overlay HWND Architecture

All bubbles are rendered inside a single full-screen WPF Window with AllowsTransparency="True" and WindowStyle="None". This window is always-on-top and passes mouse events through to underlying applications except when the cursor intersects an active bubble (using WS_EX_LAYERED + hit-testing). This avoids the DWM compositing overhead of 10–50 individual transparent HWNDs.

### Image Memory Management

When a screenshot or image is copied, Pluck immediately generates a small thumbnail (200x150 max) for bubble display and stores the original image as a compressed PNG in the SQLite BLOB column. The full-resolution image is only decoded on-demand when the user pastes it. This keeps peak RAM usage below 50 MB for a full history of 50 image items.

### Source App Detection Timing

GetForegroundWindow() is called synchronously inside the WM_CLIPBOARDUPDATE message handler, before the clipboard hook propagates. This ensures the source window handle is captured before the user switches focus to a different application. The process name and icon are extracted asynchronously to avoid blocking the UI thread.

### Animation Strategy

The floating oscillation animation is implemented as a single CompositionTarget.Rendering callback that applies a shared Math.Sin(t) offset to all bubble positions simultaneously. This avoids individual DispatcherTimer instances per bubble and ensures the animation runs on the render thread without GC allocation per frame.

# 6. Non-Functional Requirements

| Requirement | Target | Measurement Method |
|---|---|---|
| CPU usage (idle, 50 bubbles) | < 1% | Task Manager over 10-minute session |
| RAM usage (50 text items) | < 30 MB | Process Explorer private bytes |
| RAM usage (50 image items) | < 80 MB | Process Explorer private bytes |

| Requirement | Target | Measurement Method |
|---|---|---|
| Clipboard hook latency | < 50 ms from copy to bubble appear | Stopwatch instrumentation |
| Startup time (cold) | < 2 seconds to tray icon visible | Measured from process start |
| Drag-to-paste success rate | > 95% on standard apps | Manual testing matrix |
| Minimum OS version | Windows 10 1903 (Build 18362) | MSIX manifest requirement |

# 7. Settings Reference

| Setting | Type | Default | Range / Options | Description |
|---|---|---|---|---|
| Opacity | Slider | 70% | 10% – 90% | Bubble transparency level |
| Max Bubbles | Integer | 20 | 10 – 50 | Maximum simultaneous bubbles |
| Display Duration | Toggle + Integer | Off | Off \| 1–60 sec | Auto-dismiss timeout |
| Floating Animation | Toggle | On | On \| Off | Vertical oscillation effect |
| Content Display | Enum | Best Content | 3 modes | Preview richness level |
| Show Source Icon | Toggle | On | On \| Off | Source app icon visibility |
| Show Source Name | Toggle | On | On \| Off | Source app name visibility |
| Show Timestamp | Toggle | Off | On \| Off | Copy time visibility |
| Pop Effect | Toggle | On | On \| Off | Particle burst on paste |
| Global Hotkey | Key binding | Ctrl+Shift+V | Any valid combo | Opens Main Dialog |
| Launch at Startup | Toggle | Off | On \| Off | Windows startup registry |
| History Limit | Integer | 200 | 10 – 1000 | SQLite history max rows |
| Clear on Exit | Toggle | Off | On \| Off | Clears DB on app close |

# 8. Risk Register

| Risk | Severity | Likelihood | Mitigation |
|---|---|---|---|
| Multiple transparent HWNDs causing DWM lag | High | High | Single overlay HWND architecture (already planned) |
| GetForegroundWindow race condition on fast copy | Medium | Medium | Capture window handle inside WM_CLIPBOARDUPDATE handler synchronously |
| Large image items causing RAM spike | High | Medium | Thumbnail-first strategy; original stored to SQLite BLOB, loaded on demand |
| IDropTarget COM interop instability | Medium | Low | Use battle-tested SharpShell or hand-written C++/CLI wrapper with extensive testing |
| Animation causing CPU spike on low-end hardware | Medium | Medium | Single CompositionTarget.Rendering callback; disable animation on battery/low-power |
| Antivirus false positives on clipboard hook | Low | Medium | Code-sign the executable; document the hook technique in the README |

# 10. Open-Source & Portfolio Strategy

## 10.1 Repository Structure

- License: MIT — free for personal and commercial use, attribution required.
- README must include: animated screen-capture demo (GIF or MP4), feature matrix, installation steps (MSIX + manual .exe), and a brief architecture section explaining the single-HWND approach.
- GitHub Actions CI: build on every push to main; publish release assets (MSIX + portable .exe) on tag push.
- CHANGELOG.md maintained in Keep a Changelog format.
- CONTRIBUTING.md with code style guide, branch naming conventions, and PR template.

## 10.2 Monetization Options (Future)

| Model | Mechanism | Timeline |
|---|---|---|
| GitHub Sponsors | Donation page linked from README and releases | v1.0 launch |
| Freemium | Core MIT free; premium features (cloud sync, themes) as paid add-on | v2.0 |
| Commercial License | Per-seat license for enterprise deployments with support SLA | v2.0+ |

### 10.3 Portfolio Talking Points

- Demonstrates Win32 system programming: clipboard API, process enumeration, COM interop — skills rarely showcased in typical CRUD portfolio projects.
- Single-HWND overlay architecture demonstrates awareness of OS-level rendering performance constraints.
- Image memory pool demonstrates understanding of GC pressure and large-object heap management in .NET.
- The GitHub Actions CI/CD pipeline signals professional engineering practices to prospective employers.

## Appendix A — Glossary

| Term | Definition |
|---|---|
| HWND | Handle to a Windows window object — the unique identifier for any OS-level window. |
| DWM | Desktop Window Manager — the Windows compositor responsible for rendering all on-screen windows. |
| P/Invoke | Platform Invocation Services — the .NET mechanism for calling native Win32 C APIs from managed C# code. |
| COM | Component Object Model — the Windows binary interface standard used by drag-and-drop (IDropSource / IDropTarget). |
| WS_EX_LAYERED | Extended window style that enables per-pixel transparency and click-through behaviour on Windows. |
| AddClipboardFormatListener | Win32 API that registers a window to receive WM_CLIPBOARDUPDATE messages whenever the clipboard contents change. |
| CF_UNICODETEXT | Windows clipboard format identifier for Unicode text content. |
| CF_HDROP | Windows clipboard format identifier for file-drag operations. |

| Term | Definition |
| --- | --- |
| BitmapSource | WPF base class for all image sources; used to render clipboard image content without triggering GDI+ memory allocations. |

# Appendix B — References

- Microsoft Docs — Clipboard Formats: https://learn.microsoft.com/en-us/windows/win32/dataxchg/clipboard-formats
- Microsoft Docs — Drag and Drop (OLE): https://learn.microsoft.com/en-us/windows/win32/com/drag-and-drop
- WPF AllowsTransparency documentation: https://learn.microsoft.com/en-us/dotnet/api/system.windows.window.allowstransparency
- SQLite via Microsoft.Data.Sqlite: https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite
- Keep a Changelog: https://keepachangelog.com

*— End of Document —*